

vasic-digital tmux — Open Issues Tracker

Canonical source of truth for everything currently unfinished, partially validated, or at risk of violating the anti-bluff covenant (`Constitution.md` §1 + §11.4.1 through §11.4.6).

Every PASS in this codebase **MUST** carry positive evidence captured live that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks. **Tests AND HelixQA Challenges are bound equally.**

Forensic anchor — direct user mandate (verbatim, 2026-04-28 + 2026-05-07 + 2026-05-08, repeatedly reasserted from upstream vasic-digital projects):

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!"

§11.4.6 forensic anchor (verbatim, 2026-05-08):

"'LIKELY' is guessing, we **MUST NOT** have guessing, since it can be or may not be! No bluffing and uncertainty is allowed at any cost! We **MUST** always know exactly precisely what is happening exactly, in any context, under any conditions, everywhere!"

Compiled: 2026-05-08 (Phase B per-session-containerization cycle). **Author:** Engineering coordinator **Working pool source:** the items below feed the active task list directly. Each item carries a current state, the captured-evidence requirement, and a fix-direction proposal so future-self can resume cold.

Migration policy (mirrors upstream vasic-digital projects): resolved items are moved to **Fixed.md** in the same commit that closes them. **Issues.md holds OPEN / PARTIAL / BLOCKED / RUNNING / INVESTIGATED only**; once an item is closed and verified end-to-end, it migrates to `Fixed.md` and disappears from here. Never delete items outright — history matters for cold-start handover.

Document conventions

Code	Meaning
OPEN	Unfinished work; needs implementation + anti-bluff coverage
PARTIAL	Implementation exists but coverage gaps remain (positive evidence missing or environmental SKIP unresolved)
BLOCKED	Cannot progress without external dependency (host capability, third-party tool, distro support, etc.)
RUNNING	In-flight in this session (background process, ongoing test cycle)
INVESTIGATED	Forensic investigation produced findings; closure pending decision on fix scope

Status reclassification rules (§11.4.6 enforced):

- A PARTIAL may not move to closed without runtime evidence (`/sys/fs/cgroup/.../memory.max readback, systemctl status` showing scope active, `kill -9 survivor proof` — never just script exit code).
- A BLOCKED reclassifies to OPEN when the blocking dependency resolves; it must NOT skip directly to closed.
- An INVESTIGATED item must record the captured forensic trace (file path / command output / log timestamp) — never speculation ("likely" / "probably" / "appears to" — see Constitution §11.4.6).

Categories:

- **A** — Tooling / harness gaps
- **B** — Anti-bluff completeness across the existing test surface
- **C** — Per-session containerization features pending evidence
- **D** — Host-capability + topology dispatch gaps
- **E** — Documentation / Continuation drift

A. Tooling / harness gaps

(none open at this time — A1 META-MUT-001 landed in Fixed.md.)

B. Anti-bluff completeness across the existing test surface

(none open at this time — B3 P5-M20/P5-M21 escapes CLOSED in v1.0.16 [tests 49/50 + meta-test retarget], state-verified 2026-05-29 with MUTATIONS CAUGHT 45 / ESCAPED 0, and migrated to Fixed.md §B3; B1 CHAL-COVER-001, B2 TEST-AUDIT-001 also landed in Fixed.md.)

C. Per-session containerization features pending evidence

(none open at this time — C1 TMX-T5, C2 TMX-T7, C3 TMX-T8 landed in Fixed.md.)

D. Host-capability + topology dispatch gaps

(none open at this time — D1 TOPO-DISPATCH-001 landed in Fixed.md.)

E. Documentation / Continuation drift

(none open at this time; CONTINUATION.md §3 entries that resolve land in Fixed.md per Constitution §5 / §12.10.)

F. Runtime crash — operator-gated reproduction

F1. tmx session named "HelixCode" crashes the whole terminal

Status: Operator-blocked **Type:** Bug **Reported:** operator, 2026-06-13 — "Open the terminal and for terminal session choose HelixCode. It will crash the whole terminal!" Operator clarified HelixCode is a tmx SESSION NAME and the crash reproduces in iTerm2, Terminal.app, a Linux terminal, AND WezTerm (all their emulators). HelixCode is a TUI CLI agent (Claude-Code-class) run INSIDE the session.

Investigation to date (no guessing per §11.4.6 — facts only):

- A FRESH `tmx new -s HelixCode` creates a detached session (pane alive) AND a fresh interactive attach over a real PTY completes cleanly: 873 bytes, normal volume, clean detach, NO runaway redraw. Captured evidence: `docs/qa/2026-06-13-helixcode-crash/`.
- The shipped `conf` parses clean (`source-file` exit 0, no stderr); `hostname_color.sh` returns a valid colour44; the Darwin `rlimit` wrapper is benign. Cross-emulator reproduction (incl. robust WezTerm) rules out a single emulator's escape-sequence handling — the trigger is `tmx/tmux/config + state`.
- Five candidate crash vectors were each reproduced headlessly over a real PTY and **DISPROVEN as standalone causes** (forensic detail + byte counts in `docs/qa/2026-06-13-helixcode-crash/forensic.md`): (H1) pre-existing TUI + reattach repaint — fresh vs reattach streams byte-identical; (H2) `allow-passthrough` on replay — a 512 KB passthrough payload yielded a 3496-B attach stream, 0 DCS frames (tmux repaints the visible screen, never the passthrough history); (H3) `extended-keys` on — only the standard ESC [>4m modifyOtherKeys, well-formed; (H4) `automatic-rename-format .exe-strip`

— single-pass, no loop; (H5) tmx attach source-file into a live session — exit 0, empty stderr, byte-identical stream.

CONCLUSION: the crash is NOT reproducible from config + wrapper + binary state alone — it requires operator-side RUNTIME state (the real HelixCode TUI agent's own escape output under tmux, and/or a stale/wrong-arch socket or a second tmux on \$PATH, and/or a real \$TERM/size/capability mismatch). These three residuals are explicitly UNCONFIRMED — needs operator (ranked in forensic.md).

Operator-Block-Details:

- **WHAT:** run docs/qa/2026-06-13-helixcode-crash/diagnose.sh in the real crashing flow; it captures (read-only, leaves live sessions untouched) the full attach byte stream via script/tee, tmux -V, the active conf, pane_current_command, allow-passthrough/extended-keys state, the socket inventory, and whether the login shell exits. Send back the produced operator_run_<ts>/ directory.
- **WHY (self-resolution exhausted):** (a) CLI reproduction — fresh create + attach + real-PTY drive all succeed; (b) subagent forensic deep-dive — 5 hypotheses reproduced + disproven headlessly; (c) repo tooling — conf audit, color audit, source-file audit all clean; (d) captured fallback — the live HelixCode TUI agent is not installed here and the operator's stale session state cannot be fabricated; (e) external research — N/A, no published HelixCode-in-tmux crash signature.
- **UNBLOCK CONDITION:** the operator_run_<ts>/typescript byte stream shows the malformed/runaway sequence the real session emits (localise via od -c typescript | tail -40).
- **WHO:** operator (milos85vasic.3rd@gmail.com); diagnostic + forensics under docs/qa/2026-06-13-helixcode-crash/.

Last reviewed: 2026-06-13 (v1.0.21 cycle — copy/paste mouse-off default landed in Fixed.md A43; opened F1 to track the operator-gated "HelixCode" terminal crash with full forensic evidence + a ready operator diagnostic).